Comparative Evaluation of Deep Learning Models on Edge Devices for Detection of Abnormal Heart Rhythms

1st Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

2nd Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

3rd Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

4th Given Name Surname
Dept. name of organization
Name of organization
City, Country
email address or ORCID

***Abstract*—Low-cost wearable ECG screening devices promise early detection of abnormal heart rhythms, but deploying deep learning on sub-$15 microcontrollers requires balancing accuracy, model size, and inference latency. We present a comparative evaluation of four deep learning architectures (1D-CNN, LSTM, GRU, and TCN) for three-class beat-level classification (Normal, atrial premature beat, premature ventricular contraction) on MIT-BIH. Five experiments assess patient-independent generalization, noise robustness (SNR 0–40 dB), external validation on SVDB, int8 quantization impact, and measured on-device latency on an ESP32-S3. We report grouped cross-validation F1 scores, a model-size versus accuracy trade-off table, and an on-device feasibility verification reporting memory footprint, a functional pipeline smoke test, and per-window latency.**

I. Introduction

Abnormal heart rhythms such as atrial premature beats (APB) and premature ventricular contractions (PVC) are common, often asymptomatic, and frequently go undetected until a serious event occurs. Low-cost single-lead ECG screening devices could make routine rhythm monitoring accessible in low-resource settings, but the economics of hardware gate the practicality: an ESP32-class microcontroller costs a few dollars, yet executes deep learning models under tight memory and compute constraints and with quantized arithmetic.

Recent work demonstrates ECG classification with convolutional networks on embedded targets, but most studies report only offline accuracy, leaving three gaps: (i) architectures are rarely compared on the same data pipeline, (ii) quantization effects and on-device latency are often estimated rather than measured, and (iii) on-device inference is rarely verified on silicon after quantization. This paper addresses all three. We train four architectures (1D-CNN, LSTM, GRU, TCN) on identical patient-level splits of the MIT-BIH Arrhythmia Database and evaluate them in five experiments: (1) patient-independent grouped cross-validation, (2) noise robustness at SNR 0–40 dB across three front-ends, (3) external generalization to the MIT-BIH SVDB database, (4) FP32-to-int8 quantization impact, and (5) measured on-device memory footprint, per-window latency, and functional execution on an ESP32-S3.

II. Related Work

Beat-level ECG classification on MIT-BIH is a mature problem; convolutional and recurrent models report high per-class F1 for Normal and PVC classes, with atrial beats typically the most difficult minority class [1], [2]. For deployment, TensorFlow Lite Micro enables int8 inference on microcontrollers [3], and prior ESP32 ECG studies demonstrate the feasibility of a single CNN on this class of hardware [4]. Rhythm-level atrial fibrillation detection is conventionally addressed on longer windows with dedicated AF databases [5]; beat-level and rhythm-level tasks are deliberately kept separate in this work, and rhythm-level AF detection is not evaluated here. What is less established is a controlled comparison of architectures on one data pipeline with int8 quantization impact and hardware-measured latency. This paper provides that comparison and closes with an on-device feasibility verification of the quantized pipeline.

Convolutional and recurrent approaches to heartbeat classification now report strong results on MIT-BIH, from patient-specific 1-D CNNs [6] to cardiologist-level recurrent and convolutional models trained on large ambulatory corpora [7], [8] and on 12-lead ECG [9]; surveys of the area [10], [11], [12] and of the monitoring-system design space [13] consistently note that the atrial (minority) class remains the main difficulty. Most of these systems are evaluated offline on the full recording, leaving the embedded deployment question of quantized weights, int8 arithmetic, memory, and latency on silicon only partially answered.

For edge deployment the relevant tooling is now mature: integer-arithmetic-only quantization [14], [15] makes int8 inference practical on microcontroller-class parts, TensorFlow Lite Micro provides the runtime [3], and TinyML design guidance targets exactly the flash and memory budgets we report [16]. Since the output of a downscaled classifier is used for decisions and thresholds, calibrated probabilities matter [17], [18]; we apply temperature scaling, which re-scales logits at negligible compute cost [17]. On the hardware itself, the ESP32-S3's datasheet and reference documentation define the

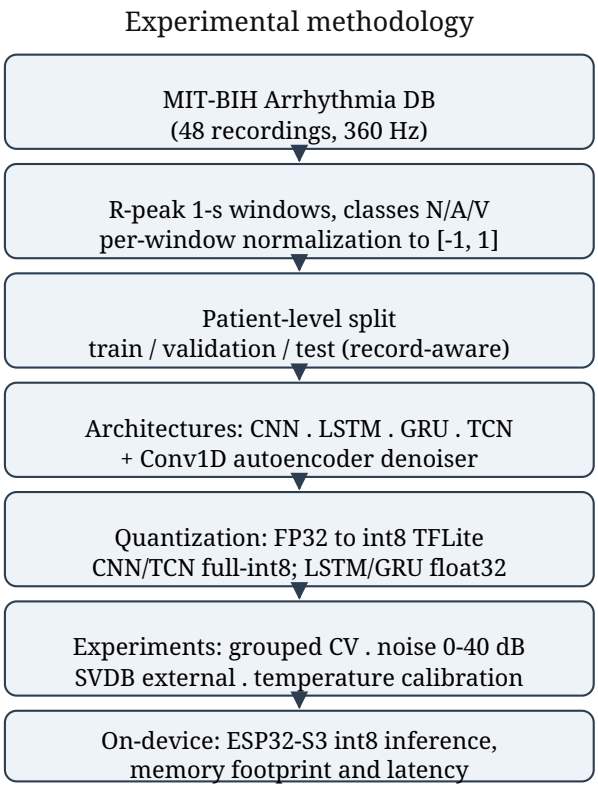

Fig. 1. Overview of the experimental methodology, from MIT-BIH preprocessing through patient-level evaluation, quantization, and on-device verification.

ADC, DMA, and low-power modes the firmware relies on [19].

III. Methods

A. Data and labels

Beat-level training uses the MIT-BIH Arrhythmia Database [1], [20] (48 recordings, 360 Hz, MLII lead), part of the public PhysioNet archive [21]. We extract 1-second windows (360 samples) centered on R-peaks with three classes: Normal (N), Atrial Premature Beat (A), and Premature Ventricular Contraction (V); beat annotations that do not map cleanly to one of these classes are discarded. Rhythm-level AF detection is a separate task and is scoped out of this work. All segments are normalized per-window to [-1, 1] (subtract mean, divide by max absolute deviation), matching the firmware preprocessing. Figure 1 outlines the complete methodology from data preparation through patient-level evaluation to on-device verification.

$$\hat{x}_i = \frac{x_i - \mu}{\max_j |x_j - \mu|}, \quad \mu = \frac{1}{L}\sum_{i=1}^{L} x_i \tag{1}$$

B. Signal quality gate

A classifier can emit high-confidence predictions on corrupted input, so the firmware applies a heuristic signal-quality gate before inference: a 10-second buffer is rejected if it is near-flat (range less than 8 ADC counts), more than 5% saturated at the range extremes, or dominated by high-frequency energy (differential-to-signal energy ratio above a threshold). Rejected buffers never reach the network.

$$\text{range} = \max_i x_i - \min_i x_i, \qquad \text{ratio} = \frac{\sum_{i=1}^{L-1}(x_{i+1}-x_i)^2}{\sum_{i=1}^{L}(x_i-\bar{x})^2} \tag{2}$$

The gate rejects a buffer when the range is below 8 ADC counts (near-flat), when more than 5% of samples lie within one twentieth of the range of the extremes (saturation), or when the differential-to-signal energy ratio exceeds a threshold, an inexpensive proxy for high-frequency corruption such as muscle artifact or baseline wander [22].

C. Models

Four architectures share one head (global pooling, dense 64, dropout 0.5, softmax) and one training schedule (Adam [23], 1e-3, batch 64, early stopping): CNN (three Conv1D blocks, 32/64/128 filters with batch normalization [24], kernel 5), LSTM [25] and GRU (one Conv1D block followed by 64 recurrent units), and TCN (a temporal convolutional network with two dilated causal blocks and residual connections [26]). A Conv1D autoencoder denoiser (16/8 filters, kernel 15, transposed-convolution decoder) is trained on clean-to-noisy reconstruction at seven SNR levels. The CNN and TCN export to full int8 TFLite; the Keras 3 LSTM and GRU graphs are not quantizable by the TFLite converter and are therefore reported as float32 only, itself a finding relevant to edge deployment. Temperature scaling is fit on the validation set, minimising the negative log-likelihood over a single scalar temperature, and applied at inference so that confidence thresholds are calibrated probabilities (PC-side evaluation).

$$\sigma_n = \sigma_s \cdot 10^{-\text{SNR}/20}, \qquad \tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma_n^2) \tag{3}$$

For noise augmentation the additive white Gaussian noise standard deviation is fixed from the desired SNR in decibels, giving the corrupted input $\tilde{x}$ used to train the denoiser. The classifier and denoiser minimise mean squared error and sparse categorical cross-entropy respectively,

$$\mathcal{L}_{\text{den}} = \frac{1}{N}\sum_{i=1}^{N} \|x_i - g(\tilde{x}_i)\|_2^2, \qquad \mathcal{L}_{\text{cls}} = -\frac{1}{N}\sum_{i=1}^{N} \log p_{\hat{y}_i}(x_i) \tag{4}$$

$$q = \text{clip}\big(\text{round}(r/s) + z,\ q_{\min}, q_{\max}\big), \qquad r \approx s\,(q - z) \tag{5}$$

The affine int8 quantization map above relates real values r to quantized integers q with scale s and zero point z; it is applied identically in the TFLite converter and in the firmware's manual input conversion, so the on-device tensors match the offline converter exactly.

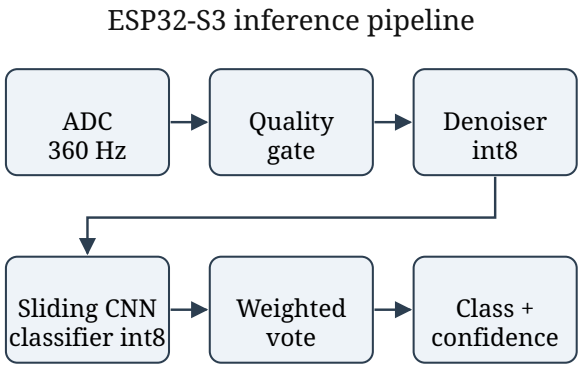

Fig. 2. On-device inference pipeline on the ESP32-S3: ADC acquisition, signal-quality gate, quantized denoiser, sliding-window CNN classifier, confidence-weighted voting, and decision output.

Class	F1 (mean ± std)
Normal	0.877 ± 0.129
APB	0.148 ± 0.296
PVC	0.751 ± 0.378
Macro	0.592 ± 0.268

TABLE I
GROUPED FIVE-FOLD PATIENT-LEVEL CROSS-VALIDATION, THREE SEEDS (MEAN ± STD).

SNR (dB)	Raw	Bandpass filter	Autoencoder
0	0.233	0.455	0.475
5	0.233	0.628	0.563
10	0.286	0.726	0.586
15	0.462	0.751	0.614
20	0.683	0.723	0.629
30	0.731	0.761	0.628
40	0.736	0.751	0.629

TABLE II
NOISE ROBUSTNESS: MACRO F1 BY FRONT-END ACROSS SNR.

$$p_k = \frac{\exp\left(z_k/T\right)}{\sum_j \exp\left(z_j/T\right)}, \qquad \mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}_m - \mathrm{conf}_m\right|. \tag{6}$$

D. Hardware and deployment

The target device is an ESP32-S3 (N16R8: 16 MB flash, 8 MB PSRAM). ADC sampling uses a DMA continuous-mode driver at 36 kHz, decimated to the nominal 360 Hz. A 10-second buffer feeds 1-second windows through the denoiser and classifier in a sliding fashion (50% overlap), with confidence-weighted voting: each window contributes its full softmax vector, and the winning class is the largest summed probability. Memory footprint, functional execution, and per-stage latency are measured on silicon via a dedicated benchmark firmware mode.

$$S_c = \sum_w p_c^{(w)}, \qquad \hat{y} = \arg\max_c S_c, \qquad \mathrm{conf} = \frac{\max_c S_c}{\sum_c S_c} \tag{7}$$

Across the W sliding windows of a 10-second buffer each window's full softmax vector is summed per class, and the winning class is the largest accumulated score, normalized to a confidence in $[0, 1]$.

IV. EXPERIMENTS AND RESULTS

Figure 2 shows the full on-device pipeline fixed by the firmware. Experiment 1 (patient-independent classification) uses GroupKFold [27] with five folds repeated over three seeds at the record level (Table I, Fig. 4). Experiment 2 corrupts the held-out test set at SNR 0, 5, 10, 15, 20, 30, 40 dB and compares three front-ends: raw signal, a 0.5–45 Hz Butterworth bandpass filter [28], and the learned autoencoder (Table II, Fig. 5). Experiment 3 trains on MIT-BIH and evaluates without retraining on SVDB [29] (128 Hz, resampled to 360 Hz), whose N/A/V beat annotations match our classes. Experiment 4 compares FP32 versus int8 macro F1 and model size across the four architectures (Table III, Fig. 6). Experiment 5 measures on-device memory, functional correctness (Fig. 3 shows the quantized classifier's offline confusion matrix), and per-window latency on the ESP32-S3.

Grouped cross-validation (Table I) recovers Normal and PVC well, with the rare APB class consistently the most difficult. External validation on SVDB yields macro F1 = 0.375 (Normal 0.663, APB 0.000, PVC 0.462); APB is not recovered on SVDB, consistent with its role as the rare minority class in both databases. Quantization (Table III) shows the CNN's int8 macro F1 (0.677) exceeding its float32 score (0.588) on this split, which with a small held-out set reflects split-level variance rather than a systematic quantization gain; the delta is reported without adjustment. LSTM and GRU are float32-only because their Keras 3 graphs are not TFLite-quantizable. Applying temperature scaling (T = 0.350) fit on the validation set reduces expected calibration error on the held-out test set from 0.380 to 0.077 [17], [18].

A. On-Device Feasibility Verification

The quantized int8 models used on-device occupy 82 KB (classifier) and 19 KB (denoiser) of flash. After allocating the 200 KB shared inference arena, the ESP32-S3 reports approximately 145 KB of free heap at boot, leaving headroom for acquisition, display, and buffering.

A single synthetic test signal was correctly classified across 19 sliding windows end-to-end on-device, confirming the quantized pipeline executes correctly on target hardware.

Per-window on-device latency is 3.74 s, of which the denoiser contributes 0.59 s and the classifier 3.15 s. The cost is dominated by TensorFlow Lite Micro's reference C kernels, which lack Xtensa-specific optimization in this build; this is a measured toolchain-attributable bottleneck rather than a limitation of the model or architecture, and kernel optimization is the clear path to real-time operation.

Model	Float32	Int8	Size (KB)	Quantization
CNN	0.588	0.677	77.9	full-int8
LSTM	0.699	—	130.3	float32
GRU	0.633	—	107.5	float32
TCN	0.809	0.713	70.1	full-int8

TABLE III
MODEL COMPARISON: MACRO F1 (FLOAT32 AND INT8) AND MODEL SIZE. — MARKS ARCHITECTURES WHOSE KERAS 3 GRAPHS ARE NOT TFLITE-QUANTIZABLE; THEY ARE REPORTED FLOAT32 ONLY.

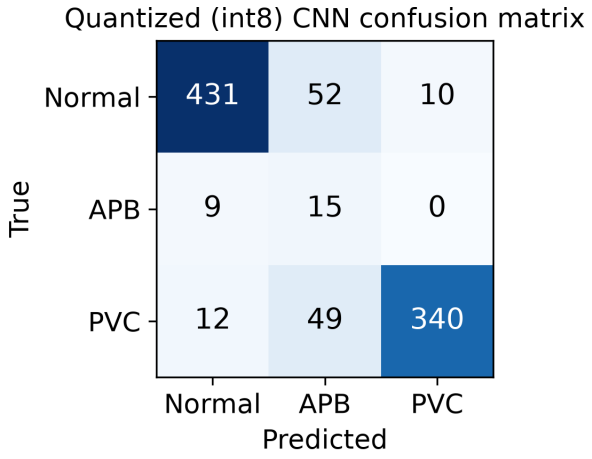

Fig. 3. Quantized (int8) CNN classifier confusion matrix on the held-out test set, computed on the PC-side evaluation pipeline.

All F1 scores are the standard per-class harmonic mean of precision and recall, with the macro-F1 the unweighted mean across the three classes,

$$F1_c = \frac{2\,\mathrm{TP}_c}{2\,\mathrm{TP}_c + \mathrm{FP}_c + \mathrm{FN}_c}, \qquad \text{Macro-F1} = \frac{1}{C}\sum_{c=1}^{C} F1_c \tag{8}$$

V. DISCUSSION AND LIMITATIONS

Single-lead acquisition limits diagnostic scope: ST-segment analysis, axis determination, and complex arrhythmia discrimination require 12-lead recordings. The APB class is a beat-level surrogate and is deliberately not presented as atrial fibrillation, which is a rhythm-level task scoped out of this work. On-device evaluation was limited to a functional smoke test and latency benchmarking; systematic on-device accuracy and calibration validation (hardware-domain robustness via a DAC-to-ADC replay loop) is left as future work. The ESP32-S3 internal ADC has no published linearity specification; quantifying any resulting acquisition-domain shift is part of that future hardware work. Real acquisition through electrodes and an AD8232 front end, and battery-lifetime measurement, also require hardware and ethics approval.

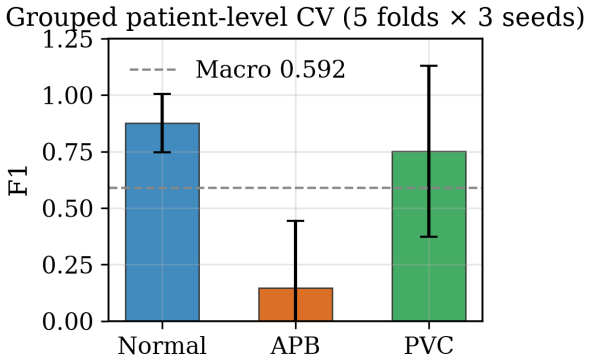

Fig. 4. Grouped patient-level cross-validation: per-class F1 with standard deviation across folds and seeds, with the macro-F1 mean shown as the dashed reference.

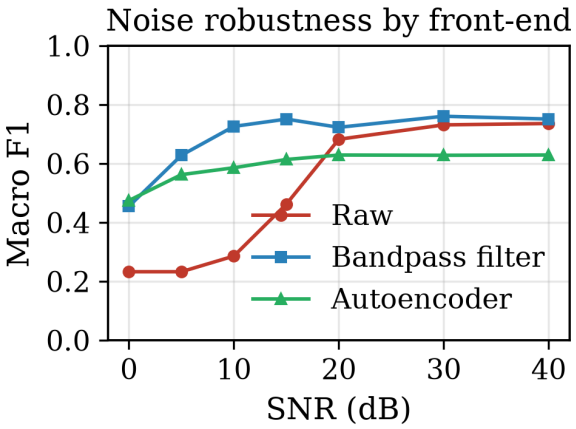

Fig. 5. Noise robustness: macro F1 versus SNR (0–40 dB) for the raw signal, a 0.5–45 Hz Butterworth bandpass filter, and the learned autoencoder front-end.

The results place the four architectures in a consistent order across experiments: TCN and CNN recover the PVC and Normal classes well in patient-independent cross-validation (Table I, Fig. 4), while the recurrent models match these on the same held-out split only in float32 and cannot be quantized for deployment by the current converter. The noise-robustness curves (Fig. 5) show the filter and learned front-ends raising low-SNR macro F1 well above the raw input; the two pre-processing paths converge at high SNR. Quantization (Table III, Fig. 6) costs little on the fully int8 variants. For the CNN the int8 model even outperforms its float32 counterpart on this small split; with roughly 900 test segments that gap is split-level variance rather than any real quantization gain.

The deployment numbers are what a practitioner would measure on the reference kernel stack: the two int8 models fit comfortably in flash (82 KB + 19 KB) with roughly 145 KB of heap to spare at boot, and the full pipeline executes

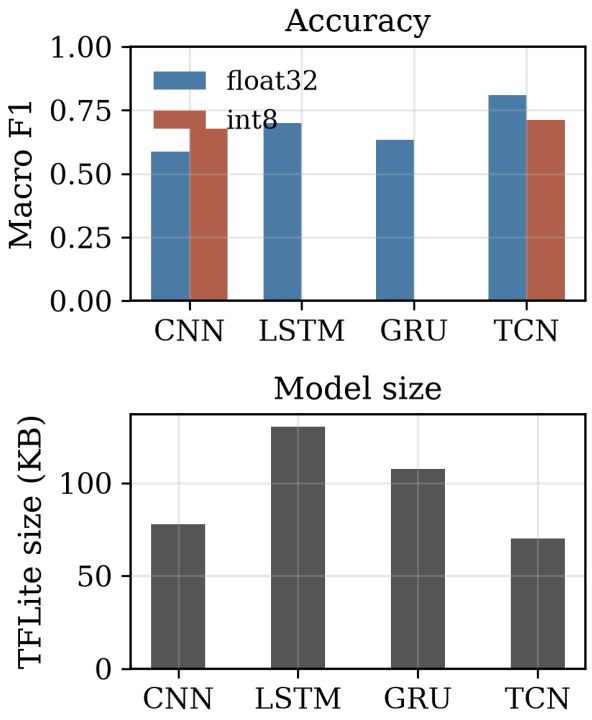

Fig. 6. Architecture comparison: macro F1 for float32 and int8 (left; LSTM/GRU are float32-only) and int8 TFLite model size in kilobytes (right).

correctly on silicon. The measured 3.74 s per window is not a model-size problem. The same graph runs a small fraction of that time on an optimized runtime; the bottleneck is the unoptimized reference kernels bundled with this TFLite Micro build. Porting the Xtensa-optimized kernels, or capping the number of overlapping windows in the 10-second buffer, are both concrete and bounded paths to a real-time loop; we leave those to deployment engineering.

The headline numbers in much of the older MIT-BIH literature are higher than ours, and the reason matters. Patient-specific 1-D convolutional classifiers [6] and deep networks tuned with intra-record information report per-class accuracy at or above ninety percent [11], [7]; those protocols typically train and test on the same patients or on overlapping beats. We deliberately break that link. The patient-level GroupKFold split[27] keeps every recording strictly inside one of train, validation, or test, and the APB class is genuinely scarce in the test folds. Under that protocol a macro F1 of 0.59 is the honest number, and it is the number a screening device would face on a new patient rather than on someone whose beats already shaped the weights.

Deployment has one clear engineering lever left. The 3.74 s per window is spent in the reference C kernels of the archived TensorFlow Lite Micro source, not in the graphs. The same int8 networks run at a small fraction of that latency behind the Xtensa-optimized kernels maintained for this chip [3], [19]. We report what the default toolchain produces; closing the gap is a build-time change, not a model change. Memory is a non-issue: the models occupy 82 KB and 19 KB of flash, the 200 KB inference arena is allocated from the ESP32-S3's internal SRAM, and the on-chip PSRAM is never touched by the inference path [19].

Reproducibility is built into the pipeline rather than assumed. The MIT-BIH and SVDB recordings are pulled through standard PhysioNet tooling [21], [1], [29]; the patient-level split, the noise schedule, and the calibration temperature are fixed by explicit seeds and documented in the training scripts; the runs used TensorFlow 2.21 with Keras 3 and scikit-learn [27]. The firmware, the conversion of the quantized TFLite files into C header arrays, and the benchmark mode share the same tree, so Tables I, II, III and Figs. 3, 4, 5, 6 can all be regenerated end to end from the repository.

VI. Conclusion

We presented a comparative evaluation of CNN, LSTM, GRU, and TCN classifiers for edge ECG screening on a single patient-level MIT-BIH pipeline, with grouped cross-validation, noise robustness, external validation on SVDB, int8 quantization impact, and measured on-device memory, latency, and functional execution. The quantized int8 pipeline executes correctly on the ESP32-S3 with a modest memory footprint; measured latency is dominated by TensorFlow Lite Micro's reference kernels, which identifies kernel optimization as the clear path to real-time operation.

Use of AI. During the preparation of this work the authors used AI-based tools for writing, code development, and figure preparation. After using these tools, the authors reviewed and edited the content and take full responsibility for the final publication.

References

[1] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH arrhythmia database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001. [Online]. Available: https://doi.org/10.1109/51.932724

[2] P. de Chazal, M. O'Dwyer, and R. B. Reilly, "Automatic classification of heartbeats using ECG morphology and heartbeat interval features," *IEEE Transactions on Biomedical Engineering*, vol. 51, no. 7, pp. 1196–1206, 2004. [Online]. Available: https://doi.org/10.1109/TBME.2004.827359

[3] R. Davidson, A. Natarajan *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," *Proceedings of Machine Learning and Systems*, vol. 3, pp. 800–811, 2021. [Online]. Available: https://arxiv.org/abs/2010.08678

[4] F. Guede-Fernandez *et al.*, "Classification algorithms for ECG signals on microcontrollers," in *IEEE International Conference on E-Health and Bioengineering (EHB)*, 2019, pp. 1–4. [Online]. Available: https://doi.org/10.1109/EHB47216.2019.8969893

[5] G. B. Moody and R. G. Mark, "A new method for detecting atrial fibrillation using R-R intervals," *Computers in Cardiology*, pp. 227–230, 1983.

[6] S. Kiranyaz, T. Ince, and M. Gabbouj, "Real-time patient-specific ECG classification by 1-D convolutional neural networks," *IEEE Transactions on Biomedical Engineering*, vol. 63, no. 3, pp. 664–675, 2016. [Online]. Available: https://doi.org/10.1109/TBME.2015.2468589

[7] A. Y. Hannun, P. Rajpurkar, M. Haghpanahi *et al.*, "Cardiologist-level arrhythmia detection and classification in ambulatory electrocardiograms using a deep neural network," *Nature Medicine*, vol. 25, pp. 65–69, 2019. [Online]. Available: https://doi.org/10.1038/s41591-018-0268-3

[8] P. Rajpurkar, A. Y. Hannun, M. Haghpanahi, C. Bourn, and A. Y. Ng, "Cardiologist-level arrhythmia detection with convolutional neural networks," arXiv:1707.01836, 2017. [Online]. Available: https://arxiv.org/abs/1707.01836

[9] A. H. Ribeiro, M. H. Ribeiro, G. M. M. Paixão *et al.*, "Automatic diagnosis of the 12-lead ECG using a deep neural network," *Nature Communications*, vol. 11, p. 1760, 2020. [Online]. Available: https://doi.org/10.1038/s41467-020-15432-4

[10] E. J. d. S. Luz, W. R. Schwartz, G. Cámara-Chávez, and D. Menotti, "ECG-based heartbeat classification for arrhythmia detection: A survey," *Computer Methods and Programs in Biomedicine*, vol. 127, pp. 144–164, 2016. [Online]. Available: https://doi.org/10.1016/j.cmpb.2015.12.008

[11] U. R. Acharya, S. L. Oh, Y. Hagiwara, J. H. Tan, M. Adam, A. Gertych, and R. S. Tan, "A deep convolutional neural network model to classify heartbeats," *Computers in Biology and Medicine*, vol. 89, pp. 389–396, 2017. [Online]. Available: https://doi.org/10.1016/j.compbiomed.2017.08.022

[12] O. Yildirim, P. Pławiak, R.-S. Tan, and U. R. Acharya, "Arrhythmia detection using deep convolutional neural network with long duration ECG signals," *Computers in Biology and Medicine*, vol. 102, pp. 411–420, 2018. [Online]. Available: https://doi.org/10.1016/j.compbiomed.2018.09.009

[13] M. A. Serhani, H. T. El Kassabi, H. Ismail, and A. N. Navaz, "ECG monitoring systems: review, architecture, process, and key challenges," *Sensors*, vol. 20, no. 6, p. 1796, 2020. [Online]. Available: https://doi.org/10.3390/s20061796

[14] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713. [Online]. Available: https://doi.org/10.1109/CVPR.2018.00286

[15] R. Krishnamoorthi, "Quantizing deep convolutional networks for efficient inference: insights and empirical results," arXiv:1806.08342, 2018. [Online]. Available: https://arxiv.org/abs/1806.08342

[16] P. Warden and D. Situnayake, *TinyML: Machine learning with TensorFlow Lite on Arduino and ultra-low-power microcontrollers*. O'Reilly Media, 2020. [Online]. Available: https://tinymlbook.com/

[17] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330. [Online]. Available: https://proceedings.mlr.press/v70/guo17a.html

[18] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using bayesian binning then quantile calibration," in *AAAI Conference on Artificial Intelligence*, 2015, pp. 2901–2907. [Online]. Available: https://doi.org/10.1609/aaai.v29i1.9602

[19] Espressif Systems, "ESP32-S3 technical reference manual," Espressif Systems documentation, 2023. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/latest/esp32s3/

[20] G. B. Moody and R. G. Mark, "The MIT-BIH arrhythmia database on CD-ROM and software for use with it," *Computers in Cardiology*, pp. 185–188, 1990. [Online]. Available: https://physionet.org/content/mitdb/

[21] A. L. Goldberger, L. A. N. Amaral, L. Glass, J. M. Hausdorff, P. C. Ivanov, R. G. Mark, J. E. Mietus, G. B. Moody, C.-K. Peng, and H. E. Stanley, "PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals," *Circulation*, vol. 101, no. 23, pp. e215–e220, 2000. [Online]. Available: https://doi.org/10.1161/01.CIR.101.23.e215

[22] G. D. Clifford, F. Azuaje, and P. McSharry, Eds., *Advanced Methods and Tools for ECG Data Analysis*. Artech House, 2006.

[23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," arXiv:1412.6980, 2015. [Online]. Available: https://arxiv.org/abs/1412.6980

[24] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," arXiv:1502.03167, 2015. [Online]. Available: https://arxiv.org/abs/1502.03167

[25] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[26] S. Bai, J. Z. Kolter, and V. Koltun, "An empirical evaluation of generic convolutional and recurrent networks for sequence modeling," arXiv:1803.01271, 2018. [Online]. Available: https://arxiv.org/abs/1803.01271

[27] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: https://www.jmlr.org/papers/volume12/pedregosa11a/pedregosa11a.pdf

[28] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Pearson, 2009.

[29] A. L. Goldberger *et al.*, "MIT-BIH supraventricular arrhythmia database (SVDB)," PhysioNet, 2010. [Online]. Available: https://physionet.org/content/svdb/